

Introduction to testing in dbt

INTERMEDIATE DBT



Mike Metzger
Data Engineer

What is a test?

- Assertions / validations of dbt objects
 - Models
 - Sources and seeds
- Used to verify data is as expected
 - Null values
 - Values in range
 - Relationships between data
 - Custom tests (will be covered later)

Test types

dbt offers three kinds of tests

1. Built-in (4 pre-defined)
2. Singular*
3. Generic*

- *will be covered later*

Built-in tests

- `unique`
 - Verify all values are unique
- `not_null`
 - Verify all values are not null
- `accepted_values`
 - Verify all values are within a specific list
 - `values: [a, b, c, d]`
- `relationships`
 - Verifies a connection to a specific table / column
 - `to: ref('table')`
 - `field: id`

Where to apply tests?

- `models/model_properties.yml`
 - File can be named anything
 - `models/schema.yml`
- Defined in the `tests:` subheading

```
version: 2

models:
  - name: taxi_rides_raw
    columns:
      - name: tpep_pickup_datetime
        tests:
          - not_null
      - name: payment_type
        tests:
          - not_null
          - accepted_values:
              values: [1, 2, 3, 4, 5, 6]
```

Running tests

- `dbt test`
 - `dbt test --select modelname`
- Verify output passes
- If failure, check against compiled sql

```
⊗ (dbt-test) Charlene:nyc_yellow_taxi mmetzger$ dbt test
04:32:39 Running with dbt=1.4.5
04:32:39 Found 3 models, 3 tests, 0 snapshots, 0 analyses, 296 macros, 0 operations, 0 seed files, 0 sources, 0 exposures, 0 metrics
04:32:39
04:32:39 Concurrency: 1 threads (target='dev')
04:32:39
04:32:39 1 of 3 START test accepted_values_taxi_rides_raw_payment_type__1__2__3__4__5__6 [RUN]
04:32:39 1 of 3 FAIL 1 accepted_values_taxi_rides_raw_payment_type__1__2__3__4__5__6 .... [FAIL 1 in 0.09s]
04:32:39 2 of 3 START test not_null_taxi_rides_raw_payment_type ..... [RUN]
04:32:39 2 of 3 PASS not_null_taxi_rides_raw_payment_type ..... [PASS in 0.01s]
04:32:39 3 of 3 START test not_null_taxi_rides_raw_tpep_pickup_datetime ..... [RUN]
04:32:40 3 of 3 PASS not_null_taxi_rides_raw_tpep_pickup_datetime ..... [PASS in 0.01s]
04:32:40
04:32:40 Finished running 3 tests in 0 hours 0 minutes and 0.15 seconds (0.15s).
```

Finding failures

1. Must find compiled SQL code
2. In the `target/compiled/projectname/models/model_properties.yml` directory
 - `target/compiled/nyc_yellow_taxi/models/model_properties.yml/`
3. Find the appropriate `.sql` file (matching the failed test)
4. Copy the contents into database client and verify where issue exists
5. Remove from data and re-run `dbt run` / `dbt test`

Let's practice!

INTERMEDIATE DBT

Creating singular tests

INTERMEDIATE DBT



Mike Metzger
Data Engineer

What is a singular test?

- Custom data test
- Written as an SQL query
 - Must return failing rows
- Defined as `.sql` file in `tests` directories



¹ Photo by Antoine Dautry on Unsplash

Example singular test

- Create a test to verify:

- `order_total` `>=` `subtotal`

```
select *  
from order  
where order_total < subtotal
```

- **Remember**, we want rows that **fail** the test if rows return, the test fails.
- Save file as
`assert_order_total_gte_subtotal.sql`

ORDER	
int	order_number
string	customer_name
datetime	order_datetime
int	subtotal
int	shipping
int	tax
int	order_total

Singular test with Jinja

- We can use Jinja in our tests
 - `ref` function
 - Others as appropriate
- dbt substitutes (replaces) output when test is run
 - Rerun the project if your dbt profile changes

```
select *  
from {{ ref('order') }}  
where order_total < subtotal
```

Test debugging

- Use a SQL editor to create the initial test query
- Place query in appropriate file
- Make sure to name the test uniquely
- Use the `dbt test --select <testname>` option
- Check any errors and update accordingly

Let's practice!

INTERMEDIATE DBT

Creating custom reusable tests

INTERMEDIATE DBT



Mike Metzger
Data Engineer

What is a reusable test?

- A test that can be reused in multiple situations
- Much like a built-in dbt test, but can check any condition
- Uses Jinja templating
- Saved as a `.sql` file in the `tests/generic` project folder
- Must add test to the `model_properties.yml` for each model that uses it



¹ Photo by Sigmund on Unsplash

Creating a reusable test

```
{% test check_gt_0(model, column_name) %}  
  
select *  
from {{ model }}  
where {{ column_name }} > 0  
  
{% endtest %}
```

1. First line (at least): `{% test testname(model, column_name) %}`
2. Add SQL query, with `{{ object }}` substitutions
3. End the file with `{% endtest %}`

Applying reusable test to model

- Add to `model_properties.yml`
- Define the objects as necessary
- The `models: name` value is the model argument
- The `columns: name` argument is the `column_name` argument

```
version: 2

models:
  - name: taxi_rides_raw
    columns:
      - name: tpep_pickup_datetime
        tests:
          - not_null
      - name: total_fare
        tests:
          - check_gt_0
```

Extra parameters

- Can add extra parameters to the test
- Similar to `accepted_values` and `relationships`
- Add as arguments to the Jinja header

```
{% test check_columns_unequal(model, column_name, column_name2) %}
```

```
select * from {{ model }}  
where {{ column_name }} = {{ column_name2 }}
```

```
{% endtest %}
```

Applying extra parameters tests

- Define like other tests
- Add the extra arguments below the test details

```
models:  
  - name: order  
    columns:  
      - name: order_time  
        tests:  
          - check_columns_unequal:  
              column_name2: shipped_time
```

Let's practice!

INTERMEDIATE DBT